Linguagem de Programação III



Profº Jânio Eduardo

janio.Eduardo@fbr.edu.br

Roteiro

Desenvolvimento Gráfico Swing:

- Familiaridade com o desenvolvimento de interfaces gráficas usando a biblioteca Swing.

- Criação de interfaces de usuário interativas e visualmente atrativas.

Introdução

Geralmente quando se está começando a programar, o desenvolvedor começa a fazer códigos que são retornados no console em formato de texto, pois muitos são códigos de aprendizagem. Mas quando é necessário desenvolver sistemas que precisam de alguma interação mais aprimorada com o usuário, utiliza-se as interfaces gráficas, então a ideia começa a evoluir, para ajudar nesse tipo de desenvolvimento usamos a interface GUI.

Conceitos básicos – Interface GUI

- Conhecida como **I**nterface **G**ráfica com **U**suário (Graphical User Interface – GUI), é onde os resultados são apresentados em modo gráfico.
- Essa interface é formada através de componentes GUI, conhecidos por **controles** ou **widgets**. Esses componentes são objetos que fazem a interação com usuário por teclado, mouse ou outros dispositivos que venham a servir para entrada de dados.

Conceitos básicos – Interface GUI

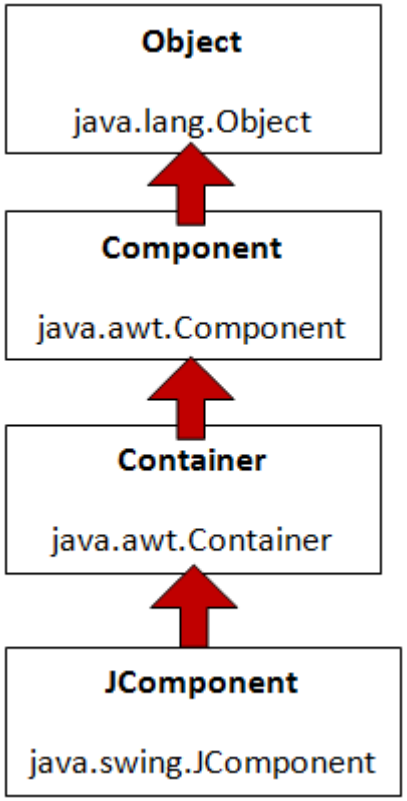
- Os componentes GUI Swing estão dentro do pacote **javax.swing** que são utilizados para construir as interfaces gráficas. Alguns componentes não são do tipo GUI Swing e sim componentes **AWT**. Antes de existir o GUI Swing, o Java tinha componentes **AWT** (Abstract Windows Toolkit) que faz parte do pacote **javax.awt**.
- A diferença entre o GUI Swing e AWT, é na aparência e comportamento dos componentes, ou seja, quando criado por AWT, a aparência e comportamento de seus componentes são diferentes para cada plataforma e enquanto feito por GUI Swing, a aparência e comportamento funcionam da mesma forma para todas as plataformas. Os componentes AWT são mais pesados, pois requerem uma interação direta com o sistema de janela local, podendo restringir na aparência e funcionalidade, ficando menos flexíveis do que os componentes GUI Swing.

Componentes mais usados

- **JLabel** - Exibe texto não editável ou ícones.
- **JTextField** – Insere dados do teclado e serve também para exibição do texto editável ou não editável.
- **JButton** – Libera um evento quando o usuário clicar nele com o mouse.
- **JCheckBox** – Especifica uma opção que pode ser ou não selecionada.
- **JComboBox** – Fornece uma lista de itens onde possibilita o usuário selecionar um item ou digitar para procurar.
- **JList** – Lista de itens onde pode ser selecionado vários itens.
- **JPanel** – É a área onde abriga e organiza os componentes inseridos.

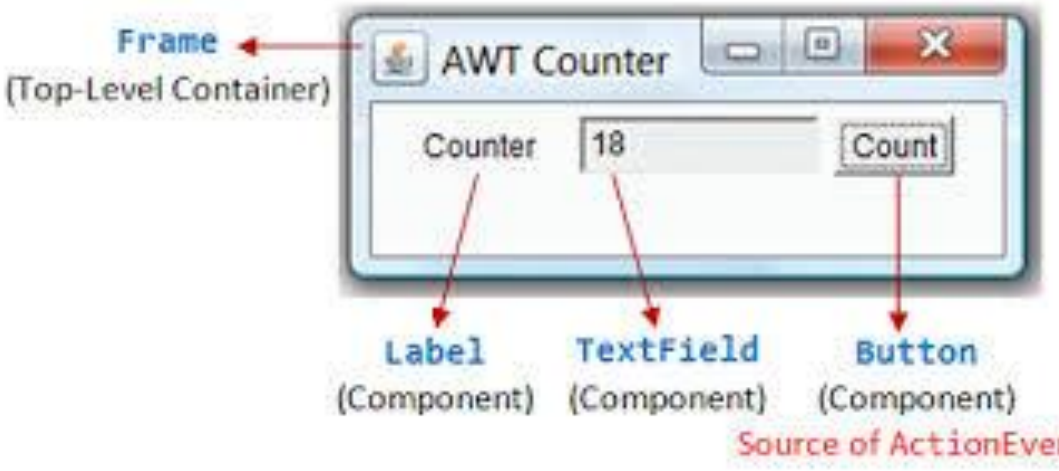
Superclasses de componentes

- Os componentes possuem classes que definem quais serão seus estados e comportamentos.



Classe Container

- Faz parte do pacote **java.awt** sendo uma subclasse de Component anexados aos **Containers.**
  Os **Containers** são janelas que podem ser usados para organizar e exibir outros componentes na tela, pelo **Container** ser um **Component**, podemos colocar Containers dentro de outros Containers para facilitar a organização.



Classe JComponent

- Está dentro do pacote **javax.swing** sendo uma subclasse de Container, mas é a superclasse que declara os atributos e comportamentos para todos os componentes Swing. Por ser a subclasse de Container, todos os componentes Swing acabam sendo Containers.
- Alguns recursos comuns do JComponent incluem aparência e comportamento plugáveis, teclas de atalho, dicas de ferramenta, suporte para tecnologias de apoio a deficientes e suporte para localização de interface com o usuário, ou seja, é essa classe que abriga os componentes principais que são usados nas janelas.
- Vamos fazer um exemplo de entrada e saída para uma interface GUI.

JOptionPane

- Encontra-se no pacote **javax.swing**, e por ser um componente faz parte da Classe JComponent, permitindo ao usuário inserir informações nas caixas de diálogos, podendo exibir informações ou avisos.
- Vamos realizar algumas saídas gráficas fazendo interações com o usuário conforme os exemplos mostrados abaixo

JOptionPane ex. 01

```java
import javax.swing.JOptionPane;

public class programaPrincipal {

    public static void main(String[] args) {

        String nome = JOptionPane.showInputDialog("Digite o nome: ");
        String sobreNome = JOptionPane.showInputDialog("Digite o sobrenome: ");

        String nomeCompleto = nome + " " + sobreNome;

        JOptionPane.showMessageDialog(null, "Nome Completo: "+nomeCompleto,"Informação",JOptionPane.INFORMATION_MESSAGE);

    }
}
```

Observe que os valores digitados na caixa de diálogo que aparece são automaticamente resultados das variáveis “nome” e “sobreNome” que estão armazenando dados conforme foi inserido através do método **showInputDialog**.
A linha que invoca o método **showMessageDialog**, imprime os resultados armazenados na variável “nomeCompleto”. Abaixo a lista de argumentos desse método.

- Argumento que marca a posição que será exibida da caixa na tela, como não estamos trabalhando com frames o padrão é null.
- Argumento que insere na barra de título o que foi digitado no caso, “Informação” .
- O último argumento é o tipo da saída da mensagem que exibe através do diálogo por meio de constante

JOptionPane ex. 02 - inteiros

```java
//segundo exemplo
import javax.swing.JOptionPane;

public class SaidaValoresInteiros {

    public static void main(String[] args) throws NumberFormatException {

        int valorA = Integer.parseInt(JOptionPane.showInputDialog("Informe o valor A: "));
        int valorB = Integer.parseInt(JOptionPane.showInputDialog("Informe o valor B: "));

        int resultado = valorA * valorB;

        JOptionPane.showMessageDialog(null,"Resultado final: "+resultado,"Resultado",JOptionPane.INFORMATION_MESSAGE);

    }
}
```

No exemplo da **Listagem 2** é necessário fazer uma conversão das Strings para tipo de dados int, que são inseridas nas variáveis “valorA” e “valorB”, pois seus tipos são inteiros e caso seja inserido outro tipo, irá gerar um erro.

JOptionPane ex. 02 - inteiros

Faculdade Brasília

```java
//Programa que gera a média
import javax.swing.JOptionPane;

public class Media_ler {
    public static void main(String[] args) {
        float nota1, nota2, calculaMedia;

        nota1 = Float.parseFloat(JOptionPane.showInputDialog ("Digite a primeira nota"));
        nota2 = Float.parseFloat (JOptionPane.showInputDialog("Digite a segunda nota"));
        calculaMedia = (nota1 + nota2) / 2;

        JOptionPane.showMessageDialog(null, "Resultado da Média = "+ calculaMedia,"Resultado", JOptionPane.WARNING_MESSAGE);
    }
}
```

Informação Diálogo de Mensagem



INFORMATION_MESSAGE -> Indica uma mensagem Informativa



ERROR _MESSAGE -> Indica um erro ao usuário



QUESTION_MESSAGE -> Mostra uma questão ao usuário



WARNING_MESSAGE -> Alerta o usuário

PLAIN_MESSAGE -> Sem ícone

Design de Interfaces Atrativas – Práticas recomendadas

- Além de conhecer os elementos de interface disponíveis, também é importante seguir algumas práticas recomendadas para criar uma interface de usuário eficaz em Java. Aqui estão algumas dicas úteis:

Design de Interfaces Atrativas – Práticas recomendadas

- Mantenha a simplicidade: Evite sobrecarregar a interface com muitos elementos e informações desnecessárias. Uma interface limpa e simplificada é mais fácil de usar e entender.
- Use consistência: Mantenha uma aparência e comportamento consistentes em seu aplicativo. Isso ajuda a criar uma experiência de usuário mais intuitiva e familiar.

Design de Interfaces Atrativas – Práticas recomendadas

- Priorize a usabilidade: Certifique-se de que todas as ações e funcionalidades do aplicativo sejam fáceis de usar e acessíveis para o usuário. Pense em como o usuário irá interagir com o aplicativo e torne essa interação o mais intuitiva possível.

Design de Interfaces Atrativas – Práticas recomendadas

- Responsividade: Considere o design responsivo ao projetar o UI em Java. Garanta que o aplicativo possa se adaptar a diferentes tamanhos de tela e dispositivos para proporcionar uma experiência consistente em todas as plataformas.

Design de Interfaces Atrativas – Práticas recomendadas

- Feedback visual: Forneça feedback visual ao usuário para informar sobre o progresso de uma ação ou para indicar o estado atual do aplicativo. Isso pode ser feito por meio de ícones, cores ou animações.

Exercício 1: Calculadora Simples

- Crie uma calculadora simples que solicita dois números ao usuário usando caixas de diálogo JOptionPane e exibe o resultado da operação selecionada (adição, subtração, multiplicação ou divisão) em outra caixa de diálogo.

Exercício 2: Conversor de Temperatura

- Desenvolva um programa que converte uma temperatura de Celsius para Fahrenheit ou vice-versa, de acordo com a escolha do usuário em uma caixa de diálogo JOptionPane. O programa deve solicitar a temperatura e a unidade de entrada, em seguida, exibir o resultado convertido em uma nova caixa de diálogo.

Exercício 3: Verificador de Idade

- Crie um aplicativo que solicita a idade do usuário em uma caixa de diálogo JOptionPane e, em seguida, exibe uma mensagem informando se o usuário é maior de idade ou menor de idade. Utilize uma caixa de diálogo JOptionPane para exibir a mensagem.

Obrigado!!!

Profº Jânio Eduardo

janio.vasconcellos@gmail.com
(61) 98451-9188